Tendências em arquiteturas e frameworks back-end

Objetivos da aula

- Nesta aula vamos analisar as principais tendências contemporâneas em arquitetura back-end.
- Você compreenderá as diferenças estruturais entre monólitos, microserviços, serverless e arquiteturas baseadas em containers.
- Também compararemos frameworks modernos sob critérios como modelo arquitetural, desempenho e curva de aprendizado.

Por que falar de tendências?

- Arquitetura de software não é estática.
- Ela evolui conforme aumentam as demandas de escalabilidade, disponibilidade e distribuição geográfica.
- Entender tendências permite tomar decisões técnicas mais conscientes.

Arquitetura Monolítica

- No modelo monolítico, toda a aplicação é desenvolvida e implantada como uma única unidade executável.
- Todos os módulos compartilham o mesmo processo e, geralmente, o mesmo banco de dados.
- Esse modelo foi predominante durante muitos anos no desenvolvimento corporativo.

Estrutura do Monólito

- Interface
- Regra de negócio
- Persistência
- Configuração

Tudo está contido no mesmo projeto e no mesmo deploy.

Vantagens do Monólito

- Arquitetura simples de iniciar
- Menor complexidade operacional
- Deploy único
- Facilidade de debug inicial

É adequado para aplicações pequenas e médias.

Desvantagens do Monólito

- À medida que cresce, o código torna-se difícil de manter.
- A escalabilidade ocorre de forma vertical, aumentando recursos da máquina.
- Uma falha pode comprometer todo o sistema.

Transição para Microserviços

- Com o aumento da complexidade dos sistemas, surgiu a necessidade de fragmentação.
- A arquitetura de microserviços propõe dividir o sistema em serviços independentes.
- Cada serviço possui responsabilidade específica e autonomia.

Arquitetura de Microserviços

- Microserviços são unidades independentes que se comunicam por APIs.
- Cada serviço pode ter seu próprio banco de dados.
- Eles são implantados e escalados separadamente.

Características dos Microserviços

- Deploy independente
- Banco por serviço
- Comunicação via HTTP ou mensageria
- Autonomia tecnológica

Essa abordagem favorece escalabilidade horizontal.

Vantagens dos Microserviços

- Escalabilidade granular
- Isolamento de falhas
- Equipes trabalhando de forma independente
- Maior flexibilidade tecnológica

São ideais para sistemas de grande porte.

Desafios dos Microserviços

- Maior complexidade operacional
- Necessidade de observabilidade avançada
- Gerenciamento de comunicação entre serviços
- Orquestração de múltiplas instâncias

Exigem maturidade arquitetural.

Acoplamento em Arquiteturas

- Acoplamento mede o nível de dependência entre componentes.
- Monólitos tendem a ter acoplamento interno elevado.
- Microserviços buscam reduzir acoplamento estrutural, mas aumentam dependências distribuídas.

Escalabilidade

- Escalabilidade vertical aumenta recursos de uma única máquina.
- Escalabilidade horizontal adiciona novas instâncias do serviço.
- Microserviços favorecem escalabilidade horizontal.

Observabilidade

- Sistemas distribuídos exigem monitoramento constante.
- Logs, métricas e rastreamento tornam-se essenciais.
- Sem observabilidade, microserviços tornam-se difíceis de gerenciar.

Containers

- Containers isolam aplicações em ambientes padronizados.
- Eles garantem que o sistema funcione da mesma forma em qualquer ambiente.
- Isso reduz problemas de inconsistência entre desenvolvimento e produção.

Docker

- O Docker popularizou o uso de containers.
- Ele permite empacotar aplicação e dependências em uma única imagem.
- Isso facilita portabilidade e consistência de execução.

Benefícios dos Containers

- Ambiente isolado
- Facilidade de deploy
- Reprodutibilidade
- Escalabilidade simplificada

São base para arquiteturas modernas.

Orquestração de Containers

- Quando há muitos containers, é necessário coordená-los.
- A orquestração gerencia criação, escala e disponibilidade.
- Ela garante alta disponibilidade do sistema.

Kubernetes

- O Kubernetes é a principal ferramenta de orquestração atualmente.
- Ele automatiza deploy, escalonamento e gerenciamento de containers.
- É amplamente utilizado em ambientes corporativos.

Serverless

- No modelo serverless, o desenvolvedor não gerencia servidores diretamente.
- A infraestrutura é abstraída pelo provedor de nuvem.
- A aplicação é executada sob demanda.

Características do Serverless

- Execução orientada a eventos
- Escalabilidade automática
- Cobrança por uso
- Infraestrutura invisível

Esse modelo reduz preocupação operacional.

Vantagens do Serverless

- Alta escalabilidade automática
- Redução de custos em baixa demanda
- Menor responsabilidade de infraestrutura

Ideal para cargas intermitentes.

Limitações do Serverless

- Dependência do provedor
- Possível latência inicial
- Limitações de execução

Nem todos os sistemas se adaptam bem a esse modelo.

Comparação: Monólito vs Microserviços

- Monólito é simples e centralizado.
- Microserviços são distribuídos e independentes.
- A escolha depende do tamanho do sistema e maturidade da equipe.

Comparação: Containers vs Serverless

- Containers oferecem maior controle sobre ambiente.
- Serverless abstrai completamente a infraestrutura.
- Um prioriza controle, o outro conveniência

Frameworks Modernos

- Frameworks influenciam diretamente a arquitetura adotada.
- Eles podem facilitar monólitos ou microsserviços.
- A escolha impacta desempenho e produtividade.

Spring Boot

- O Spring Boot é amplamente usado em aplicações corporativas.
- Ele suporta tanto monólitos quanto microserviços.
- Possui ecossistema robusto e maduro.

Micronaut

- O Micronaut é voltado para aplicações leves e rápidas.
- Foi projetado com foco em inicialização rápida e baixo consumo de memória.
- É adequado para ambientes cloud-native.

Quarkus

- O Quarkus é otimizado para containers e ambientes Kubernetes.
- Possui tempo de inicialização reduzido.
- É frequentemente associado a arquiteturas serverless.

Express.js

- O Express.js é um framework minimalista para Node.js.
- É leve e flexível.
- Muito utilizado para APIs REST e microsserviços.

Curva de Aprendizado

- Spring Boot possui curva moderada a alta.
- Micronaut e Quarkus exigem compreensão de conceitos modernos de JVM.
- Express.js é mais simples para iniciantes em JavaScript.

Performance

- Frameworks mais leves tendem a consumir menos memória.
- Quarkus e Micronaut são otimizados para ambientes cloud.
- Spring Boot prioriza robustez e ecossistema.

Ecossistema

- Spring possui ecossistema extremamente amplo.
- Express.js depende fortemente de bibliotecas externas.
- Micronaut e Quarkus estão em crescimento.

Cloud Native

- Arquiteturas modernas são projetadas para nuvem.
- Containers e microsserviços se alinham a esse modelo.
- Frameworks recentes já nascem com esse foco.

DevOps e Arquitetura

- DevOps e ArquiteturaTendências arquiteturais estão ligadas à cultura DevOps.
- Automação de deploy e integração contínua são fundamentais.
- Arquitetura moderna exige pipeline estruturado.

Escolha Arquitetural

- Não existe arquitetura universalmente superior.
- A decisão depende do contexto, equipe e objetivo do sistema.
- Arquitetura deve resolver problemas reais, não seguir modismos.

Complexidade vs Benefício

- Microserviços aumentam complexidade operacional.
- Monólitos reduzem complexidade inicial.
- Equilíbrio é essencial na decisão arquitetural.

Tendência de Mercado

- Grandes empresas utilizam microsserviços e containers.
- Startups frequentemente iniciam com monólitos.
- A migração ocorre conforme crescimento da demanda.

Síntese Comparativa

- Monólito → Simplicidade
- Microserviços → Escalabilidade distribuída
- Containers → Padronização de ambiente
- Serverless → Infraestrutura abstraída

Frameworks modernos suportam esses diferentes modelos.

Encerramento

- Você analisou as principais tendências em arquitetura back-end contemporânea.
- Compreendeu diferenças entre monólitos, microserviços, containers e serverless.
- Também comparou frameworks populares sob critérios estruturais e de desempenho.

Com esse panorama, torna-se possível avaliar arquiteturas com visão crítica e estratégica.

PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.
Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras
modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.